ML Model Deployment Strategies and Model Serving using FastAPI

Name: R. Sai Grishma
Registration No.: 231FA04D37
Section: 09

1. Objective

The objective of this assignment is to train a classification model on the Kaggle Titanic training dataset, save the trained preprocessing and prediction pipeline, demonstrate online, batch and streaming inference, and expose predictions through a FastAPI REST API.

2. Dataset and Preprocessing

The dataset used was the official Titanic competition train.csv file. The target column was Survived. The model used passenger class, sex, age, number of siblings/spouses aboard, number of parents/children aboard, fare, and embarkation port as input features. Missing numeric values were imputed with the median and missing categorical values with the most frequent value. Numeric features were standardized and categorical features were one-hot encoded as part of the pipeline.

3. Training and Evaluation

A Random Forest classifier was trained within a scikit-learn Pipeline. Ten percent of the labelled records were reserved before model development for the inference demonstrations. The remaining records were split into training and testing portions using stratification. In the recorded run, the held-out test accuracy was 83.23%. The fitted pipeline was saved using Joblib as model.pkl and reused for the inference demonstrations.

4. Inference Strategies

Online inference: One reserved passenger record was passed to the saved pipeline and a survival prediction was displayed.

Batch inference: Twenty reserved passenger records were predicted together. The results were saved to batch_predictions.csv with predicted class and a readable prediction label.

Streaming inference: Ten reserved records were processed one at a time, with a one-second pause between records to simulate a stream of incoming data.

5. FastAPI Model Serving

The FastAPI application loaded model.pkl and provided GET /health and POST /predict endpoints. Pydantic was used to validate the passenger request fields. During testing, the health endpoint returned HTTP 200 and indicated that the model was loaded. A valid prediction request returned HTTP 200 with a prediction and label. An invalid request using Pclass = 8 returned HTTP 422 because the accepted passenger-class range is 1 to 3. The Python client also received successful HTTP 200 responses from the health and prediction endpoints.

6. Observations

1. How does Online Inference differ from Batch Inference?

Online inference predicts one record per request and is useful when a result is needed immediately. Batch inference processes multiple records together and is suitable for bulk predictions and reports.

2. What is the difference between Streaming Inference and Batch Inference?

Streaming inference processes each record as it arrives. Batch inference collects a group of records and processes them together.

3. Why is FastAPI used for model serving?

FastAPI makes the model available through HTTP endpoints, supports request validation with Pydantic, and provides interactive API documentation through Swagger UI.

4. Why should the same trained model be reused for all inference strategies?

Using the same saved pipeline keeps preprocessing and predictions consistent across online, batch, streaming, and API use. It also avoids unnecessary retraining and supports reproducibility.

7. Conclusion

The assignment demonstrated model training and evaluation, saving the trained pipeline, generating predictions through three inference strategies, and serving predictions through FastAPI. The recorded test accuracy was 83.23%. API testing confirmed successful valid requests and rejection of invalid passenger data through Pydantic validation.

Note: Confirm that the accuracy value in this report matches the final output from your own Colab run before submitting.